

LAB REPORT 04

SWITCH, BREAK, AND CONTINUE STATEMENTS

1. OBJECTIVE

- To understand and use the switch statement as an alternative to multiple if-else statements.
- To apply the break statement to control and terminate program flow.
- To use the continue statement inside loops.
- To develop simple, interactive menu-driven programs.

2. THEORETICAL BACKGROUND

The Switch Statement. The switch statement selects one block of code from multiple options based on the value of a constant integral expression. It performs one of many different actions depending on the evaluated integer or character value.

The Break Statement. The break statement is used inside the switch to terminate a statement sequence. When a break is reached, the switch terminates, preventing the flow of control from 'falling through' to subsequent cases.

The Default Section. The default section is an optional block that executes if none of the preceding case expressions match the evaluated switch expression. It acts as a fallback or error-handler for invalid inputs.

3. SOFTWARE USED

- IDE: DEV-C++ IDE / VS Code
- Compiler: C++ Compiler

4. EXPERIMENTAL TASKS

Task 1: Day Number to Day Name

Write a C++ program that prompts the user to enter a day's number (1-7) and uses a switch statement to output the corresponding day's name. A break statement is used to end each case.

```
#include <iostream>
using namespace std;

int main() {
    int day;
    cout << "Enter a day number (1-7): ";
    cin >> day;

    switch(day) {
        case 1: cout << "Monday" << endl; break;
        case 2: cout << "Tuesday" << endl; break;
        case 3: cout << "Wednesday" << endl; break;
        case 4: cout << "Thursday" << endl; break;
        case 5: cout << "Friday" << endl; break;
        case 6: cout << "Saturday" << endl; break;
        case 7: cout << "Sunday" << endl; break;
        default: cout << "Invalid day number!" << endl;
    }
    return 0;
}
```

Task 2: Grade Evaluation

Write a C++ program using a switch statement that takes a character grade (A, B, C, D, F) from the user and prints the corresponding performance remark. It displays 'Invalid Grade' for unrecognized inputs.

```
#include <iostream>
using namespace std;

int main() {
    char grade;
    cout << "Enter your grade (A, B, C, D, F): ";
    cin >> grade;

    switch(grade) {
        case 'A': cout << "Excellent" << endl; break;
        case 'B': cout << "Very Good" << endl; break;
        case 'C': cout << "Good" << endl; break;
        case 'D': cout << "Pass" << endl; break;
        case 'F': cout << "Fail" << endl; break;
        default: cout << "Invalid Grade" << endl;
    }
    return 0;
}
```

Task 3: Menu-Driven Unit Converter

Write a menu-driven program that allows the user to select from a list of conversions (Kilometers to Meters, Celsius to Fahrenheit, etc.). Use a switch statement to execute the chosen conversion formula.

```

#include <iostream>
using namespace std;

int main() {
    int choice;
    double value, result;

    cout << "--- CONVERSION MENU ---" << endl;
    cout << "1. Kilometers to Meters" << endl;
    cout << "2. Meters to Centimeters" << endl;
    cout << "3. Kilograms to Grams" << endl;
    cout << "4. Celsius to Fahrenheit" << endl;
    cout << "Enter your choice (1-4): ";
    cin >> choice;

    switch(choice) {
        case 1:
            cout << "Enter Kilometers: "; cin >> value;
            cout << value << " km = " << (value * 1000) << " m" << endl;
            break;
        case 2:
            cout << "Enter Meters: "; cin >> value;
            cout << value << " m = " << (value * 100) << " cm" << endl;
            break;
        case 3:
            cout << "Enter Kilograms: "; cin >> value;
            cout << value << " kg = " << (value * 1000) << " g" << endl;
            break;
        case 4:
            cout << "Enter Celsius: "; cin >> value;
            cout << value << " C = " << ((value * 9.0/5.0) + 32) << " F" <<
endl;
            break;
        default:
            cout << "Invalid choice!" << endl;
    }
    return 0;
}

```

5. CONCLUSION

This lab demonstrated the efficiency of the switch statement for creating menu-driven programs and handling multiple discrete conditions. Compared to deep nested if-else ladders, the switch statement provides a cleaner, more readable syntax. The crucial role of the break statement was observed to prevent logical fall-through errors, ensuring only the selected block of code is executed.